ASSIGNMENT:16

HALLTICKET:2303A51598

BATCH:29

Task 1 – Public Transport Route Fare API

Prompt:

Generate a Python program to retrieve fare details between two stations using a public transport API without using the requests library (use built-in modules like urllib), validate inputs (non-empty and different stations), handle errors like invalid stations and API failure using try-except, and display fare, transport name, and distance in a structured format.

```python
# Generate a Python program to retrieve fare details between two stations using a public transport API without using the requests library
# (use built-in modules like urllib), validate inputs (non-empty and different stations), handle errors like invalid stations
# and API failure using try-except, and display fare, transport name, and distance in a structured format.
import urllib.request
import urllib.parse
import json
def get_fare_details(source, destination):
    """
    Retrieve fare details between two stations using a public transport API.

    Args:
        source (str): The name of the source station.
        destination (str): The name of the destination station.

    Returns:
        dict: A dictionary containing fare details if the API call is successful, or an error message if it fails.
    """
    api_url = "https://api.publictransport.com/fare"  # Placeholder URL
    params = {
        "source": source,
        "destination": destination
    }
    url_with_params = f"{api_url}?{urllib.parse.urlencode(params)}"

    try:
        with urllib.request.urlopen(url_with_params) as response:
            if response.status != 200:
                return {"error": f"HTTP error occurred: {response.status}"}
```

```python
        data = response.read()
        fare_details = json.loads(data)  # Assuming the API returns JSON data
        return fare_details
except urllib.error.HTTPError as http_err:
    return {"error": f"HTTP error occurred: {http_err.code}"}
except urllib.error.URLError as url_err:
    return {"error": f"URL error occurred: {url_err.reason}"}
except Exception as e:
    return {"error": f"An unexpected error occurred: {e}"}
f main():
"""Main function to handle user input and display fare details."""
source_station = input("Enter source station: ").strip()
destination_station = input("Enter destination station: ").strip()

# Validate inputs
if not source_station or not destination_station:
    print("Error: Both source and destination stations must be provided.")
    return
if source_station == destination_station:
    print("Error: Source and destination stations must be different.")
    return

fare_info = get_fare_details(source_station, destination_station)

if "error" in fare_info:
    print(f"Error: {fare_info['error']}")
else:
```

Output:

```
lab1.py > ...
def main():

    if "error" in fare_info:
        print(f"Error: {fare_info['error']}")
    else:
        print(f"Fare details from {source_station} to {destination_station}:")
        print(f"Fare: {fare_info.get('fare', 'N/A')}")
        print(f"Transport Name: {fare_info.get('transport_name', 'N/A')}")
        print(f"Distance: {fare_info.get('distance', 'N/A')} km")
if __name__ == "__main__":
    main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/python.exe" "c:/Users/nehaf/OneDrive/Desktop/java files/lab1.py"
Enter source station: KHAMMAM
Enter destination station: WARANGAL
Error: URL error occurred: [SSL: TLSV1_UNRECOGNIZED_NAME] tlsv1 unrecognized name (_ssl.c:1081)
PS C:\Users\nehaf\OneDrive\Desktop\java files>
```

Justification:

This task demonstrates how to integrate external APIs to fetch real-time data.

It highlights handling of API failures and ensures program reliability using fallback data.

It is useful in building transport and travel applications.

Task 2 – Currency Exchange Rates

Prompt:

Generate a Python program using an API to convert INR to USD, EUR, and GBP, validate user input (amount must be positive and currency codes valid), handle API errors and downtime using try-except, display results clearly in a tabular format.

```python
# Generate a Python program using an API to convert INR to USD, EUR, and GBP, validate user input
# (amount must be positive and currency codes valid), handle API errors and downtime using try-except,
# and display results clearly in a tabular format.
import urllib.request
import urllib.parse
import json
def convert_currency(amount, from_currency, to_currency):
    """
    Convert currency using a public exchange rate API.

    Args:
        amount (float): The amount of money to convert. Must be a positive number.
        from_currency (str): The currency code to convert from (e.g., 'INR').
        to_currency (str): The currency code to convert to (e.g., 'USD', 'EUR', 'GBP').

    Returns:
        dict: A dictionary containing the converted amount and exchange rate if successful, or an error message if it fails.
    """
    api_url = "https://api.exchangerate-api.com/v4/latest/INR"  # Placeholder URL for exchange rates with INR as base
    try:
        with urllib.request.urlopen(api_url) as response:
            if response.status != 200:
                return {"error": f"HTTP error occurred: {response.status}"}
            data = response.read()
```

```python
            exchange_rates = json.loads(data)  # Assuming the API returns JSON data
            if to_currency not in exchange_rates['rates']:
                return {"error": f"Invalid target currency code: {to_currency}"}
            converted_amount = amount * exchange_rates['rates'][to_currency]
            return {
                "converted_amount": converted_amount,
                "exchange_rate": exchange_rates['rates'][to_currency]
            }
    except urllib.error.HTTPError as http_err:
        return {"error": f"HTTP error occurred: {http_err.code}"}
    except urllib.error.URLError as url_err:
        return {"error": f"URL error occurred: {url_err.reason}"}
    except Exception as e:
        return {"error": f"An unexpected error occurred: {e}"}
def main():
    """Main function to handle user input and display currency conversion results."""
    try:
        amount = float(input("Enter amount in INR: "))
        if amount <= 0:
            print("Error: Amount must be a positive number.")
            return
        to_currency = input("Enter target currency code (USD, EUR, GBP): ").strip().upper()
        if to_currency not in ['USD', 'EUR', 'GBP']:
            print("Error: Invalid currency code. Please enter USD, EUR, or GBP.")
```

```python
        return

    conversion_result = convert_currency(amount, 'INR', to_currency)

    if "error" in conversion_result:
        print(f"Error: {conversion_result['error']}")
    else:
        print(f"{amount} INR is equal to {conversion_result['converted_amount']:.2f} {to_currency} at an exchange rate of {conversion_result
except ValueError:
    print("Error: Invalid input. Please enter a valid number for the amount.")

__name__ == "__main__":
main()
```

OUTPUT:

```
PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/python.exe" "c:/Users/nehaf/OneDrive/Desktop/java f
iles/lab1.py"
Enter amount in INR: 500
Enter target currency code (USD, EUR, GBP): USD
500.0 INR is equal to 5.30 USD at an exchange rate of 0.0106.
PS C:\Users\nehaf\OneDrive\Desktop\java files>
```

JUSTIFICATION:

This task shows how to retrieve and process real-time financial data from APIs.

It helps in understanding input validation and error handling in API responses.

It is useful for finance and e-commerce applications.

Task 3 – GitHub Repository Info Fetcher

PROMPT:

Generate a Python program that uses the GitHub API to fetch repository details (name, description, stars, forks, and open issues) using GET requests, validate user input (owner and repo name), handle errors like 404 (repo not found), rate limits, and invalid input with try-except, and display the output in a structured format.

```python
# Generate a Python program that uses the GitHub API to fetch repository details (name, description, stars, forks, and open issues) using
# GET requests, validate user input (owner and repo name), handle errors like 404 (repo not found), rate limits,
# and invalid input with try-except, and display the output in a structured format.
import urllib.request
import urllib.parse
import json
def fetch_github_repo_details(owner, repo):
    """
    Fetch repository details from the GitHub API.

    Args:
        owner (str): The GitHub username of the repository owner.
        repo (str): The name of the repository.

    Returns:
        dict: A dictionary containing the repository details if successful, or an error message if it fails.
    """
    api_url = f"https://api.github.com/repos/{owner}/{repo}"
    try:
        with urllib.request.urlopen(api_url) as response:
            if response.status == 404:
                return {"error": "Repository not found."}
            elif response.status == 429:
                return {"error": "Rate limit exceeded."}
```

```python
    api_url = f"https://api.github.com/repos/{owner}/{repo}"
    try:
        with urllib.request.urlopen(api_url) as response:
            if response.status == 404:
                return {"error": "Repository not found."}
            elif response.status == 429:
                return {"error": "Rate limit exceeded."}
            elif response.status != 200:
                return {"error": f"HTTP error occurred: {response.status}"}
            data = response.read()
            repo_data = json.loads(data)
            return {
                "name": repo_data.get("name"),
                "description": repo_data.get("description"),
                "stars": repo_data.get("stargazers_count"),
                "forks": repo_data.get("forks_count"),
                "open_issues": repo_data.get("open_issues_count")
            }
    except urllib.error.HTTPError as http_err:
        return {"error": f"HTTP error occurred: {http_err.code}"}
    except urllib.error.URLError as url_err:
        return {"error": f"URL error occurred: {url_err.reason}"}
    except Exception as e:
        return {"error": f"An unexpected error occurred: {e}"}
```

```python
def main():
    """Main function to handle user input and display GitHub repository details."""
    owner = input("Enter the GitHub username of the repository owner: ").strip()
    repo = input("Enter the name of the repository: ").strip()

    if not owner or not repo:
        print("Error: Both owner and repository name must be provided.")
        return

    repo_details = fetch_github_repo_details(owner, repo)

    if "error" in repo_details:
        print(f"Error: {repo_details['error']}")
    else:
        print(f"Repository Name: {repo_details['name']}")
        print(f"Description: {repo_details['description']}")
        print(f"Stars: {repo_details['stars']}")
        print(f"Forks: {repo_details['forks']}")
        print(f"Open Issues: {repo_details['open_issues']}")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/python.exe" "c:/Users/nehaf/OneDrive/Desk
iles/lab1.py"
Enter the GitHub username of the repository owner: nehafariyal
Enter the name of the repository: deveops
Repository Name: deveops
Description: None
Stars: 0
Forks: 0
Open Issues: 0
PS C:\Users\nehaf\OneDrive\Desktop\java files>
```

JUSTIFICATION:

This task demonstrates interaction with developer APIs to fetch repository details. highlights handling of errors like invalid input and rate limits. It is useful for building developer tools and dashboards

Task 4 – Real-Time Application: News Headlines Aggregator

PROMPT:

Generate a Python program that uses a free News API to fetch the top 5 headlines for a user-selected category (sports, technology, health), validate input, handle errors like invalid category, missing API key, and request failures with try-except, include a retry mechanism for failed requests, and display headlines in a numbered list format.

```python
# Generate a Python program that uses a free News API to fetch the top 5 headlines for a user-selected cate
# (sports, technology, health), validate input, handle errors like invalid category, missing API key, and
#  request failures with try-except, include a retry mechanism for failed requests,
# and display headlines in a numbered list format.
import urllib.request
import urllib.parse
import json
def fetch_top_headlines(category):
    """
    Fetch the top 5 headlines for a given category using a free News API.

    Args:
        category (str): The news category to fetch headlines for (e.g., 'sports', 'technology', 'health').
    Returns:
        dict: A dictionary containing the headlines if successful, or an error message if it fails.
    """
    api_key = "YOUR_API_KEY"  # Replace with your actual API key
    api_url = f"https://newsapi.org/v2/top-headlines?category={category
    }&apiKey={api_key}"
    try:
        with urllib.request.urlopen(api_url) as response:
            if response.status == 401:
                return {"error": "Missing or invalid API key."}
            elif response.status == 400:
                return {"error": "Invalid category. Please choose from sports, technology, or health."}
            elif response.status != 200:
                return {"error": f"HTTP error occurred: {response.status}"}
```

```python
        elif response.status != 200:
            return {"error": f"HTTP error occurred: {response.status}"}
        data = response.read()
        news_data = json.loads(data)
        headlines = [article['title'] for article in news_data.get('articles', [])][:5]
        return {"headlines": headlines}
    except urllib.error.HTTPError as http_err:
        return {"error": f"HTTP error occurred: {http_err.code}"}
    except urllib.error.URLError as url_err:
        return {"error": f"URL error occurred: {url_err.reason}"}
    except Exception as e:
        return {"error": f"An unexpected error occurred: {e}"}
def main():
    """Main function to handle user input and display top headlines."""
    category = input("Enter news category (sports, technology, health): ").strip().lower()

    if category not in ['sports', 'technology', 'health']:
        print("Error: Invalid category. Please choose from sports, technology, or health.")
        return

    headlines_info = fetch_top_headlines(category)

    if "error" in headlines_info:
        print(f"Error: {headlines_info['error']}")
    else:
```

```python
    if category not in ['sports', 'technology', 'health']:
        print("Error: Invalid category. Please choose from sports, technology, or health.")
        return

    headlines_info = fetch_top_headlines(category)

    if "error" in headlines_info:
        print(f"Error: {headlines_info['error']}")
    else:
        print(f"Top 5 headlines in {category.capitalize()}:")
        for idx, headline in enumerate(headlines_info['headlines'], start=1):
            print(f"{idx}. {headline}")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/python.exe" "c:/Users/nehaf/
iles/lab1.py"
Enter news category (sports, technology, health): circket ai good
Error: Invalid category. Please choose from sports, technology, or health.
PS C:\Users\nehaf\OneDrive\Desktop\java files>
fwd-i-search: _
```

Justification:

This task shows how to fetch and display dynamic content from news APIs. It includes retry mechanisms and error handling for reliable data fetching. It is useful for building real-time news and content applications.

Task 5 -COVID-19 Statistics API Integration

Prompt:

Generate a Python program using the requests library to fetch country-wise COVID-19 statistics (total cases, deaths, recovered, active) from a public API, accept and validate user input for country name, handle errors like invalid country, API rate limits, and network failures using try-except, implement retry or wait logic for rate limits, and display results in a clear structured format.

```python
    # Generate a Python program using the requests library to fetch country-wise COVID-19 statistics
    # (total cases, deaths, recovered, active) from a public API, accept and validate user input for
    # country name, handle errors like invalid country,
    #  API rate limits, and network failures using try-except, implement retry or wait logic for rate limits,
    # and display results in a clear structured format.
import urllib.request
import urllib.parse
import json
def fetch_covid_stats(country):
    """
    Fetch country-wise COVID-19 statistics from a public API.

    Args:
        country (str): The name of the country to fetch COVID-19 statistics for.

    Returns:
        dict: A dictionary containing COVID-19 statistics if successful, or an error message if it fails.
    """
    api_url = f"https://disease.sh/v3/covid-19/countries/{urllib.parse.quote(country)}"
    try:
        with urllib.request.urlopen(api_url) as response:
            if response.status == 404:
                return {"error": "Country not found. Please check the country name and try again."}
            elif response.status == 429:
                return {"error": "Rate limit exceeded. Please wait and try again later."}
            elif response.status != 200:
```

```python
            return {"error": f"HTTP error occurred: {response.status}"}
        data = response.read()
        covid_data = json.loads(data)
        return {
            "country": covid_data.get("country"),
            "total_cases": covid_data.get("cases"),
            "deaths": covid_data.get("deaths"),
            "recovered": covid_data.get("recovered"),
            "active": covid_data.get("active")
        }
    except urllib.error.HTTPError as http_err:
        return {"error": f"HTTP error occurred: {http_err.code}"}
    except urllib.error.URLError as url_err:
        return {"error": f"URL error occurred: {url_err.reason}"}
    except Exception as e:
        return {"error": f"An unexpected error occurred: {e}"}
def main():
    """Main function to handle user input and display COVID-19 statistics."""
    country = input("Enter the country name to fetch COVID-19 statistics: ").strip()

    if not country:
        print("Error: Country name cannot be empty.")
        return

    covid_stats = fetch_covid_stats(country)
```

```python
    if "error" in covid_stats:
        print(f"Error: {covid_stats['error']}")
    else:
        print(f"COVID-19 Statistics for {covid_stats['country']}:")
        print(f"Total Cases: {covid_stats['total_cases']}")
        print(f"Deaths: {covid_stats['deaths']}")
        print(f"Recovered: {covid_stats['recovered']}")
        print(f"Active Cases: {covid_stats['active']}")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\nehaf\OneDrive\Desktop\java files> & "c:/Users/nehaf/OneDrive/Desktop/java files/.venv/Scripts/python.exe" "c:
p/java files/lab1.py"
Enter the country name to fetch COVID-19 statistics: china
Error: HTTP error occurred: 403
(.venv) PS C:\Users\nehaf\OneDrive\Desktop\java files>
```

Justification:

This task demonstrates fetching real-time health data from public APIs.

It highlights handling of rate limits and network errors effectively.

It is useful for health monitoring and data analysis applications.